Experimental Performance Analysis of Quicksort Pivot Selection Strategies

Algorithm Design & Analysis (CSE-323) — Assignment 01

MD Mominul Islam
Registration No: 2024331022
Department of Computer Science and Engineering
Shahjalal University of Science and Technology, Sylhet, Bangladesh

Abstract

Quicksort remains a cornerstone of sorting algorithms due to its average-case O(N log N) complexity and low memory footprint. However, its performance is highly sensitive to the choice of the pivot element. Naïve pivot selections can cause the algorithm to degenerate into an $O(N^2)$ quadratic time complexity on pre-sorted inputs. This paper conducts a rigorous empirical investigation of four pivot selection strategies: First Element, Last Element, Middle Element, and Random Element. We benchmark these strategies using arrays ranging from 10 to 5,000 elements under three initial conditions: random, sorted ascending, and sorted descending. Our empirical findings confirm the theoretical predictions: First and Last Element strategies suffer catastrophic performance degradation to $O(N^2)$ on sorted arrays, whereas the Middle and Random Element strategies consistently maintain O(N log N) complexity. The C++ implementation utilizes C. A. R. Hoare's partitioning scheme with standard recursion, showing that a stack depth of up to 5,000 is safely within Windows' default stack limits.

1. Introduction

Sorting is a fundamental problem in computer science, and C. A. R. Hoare's Quicksort is widely regarded as one of the most efficient algorithms for general-purpose sorting. Utilizing a divide-and-conquer strategy, Quicksort partitions an array around a chosen 'pivot' element such that elements smaller than the pivot are placed to its left and larger elements to its right. The sub-arrays are then recursively sorted. While the average-case performance of Quicksort is O(N log N), its worst-case behavior is $O(N^2)$. This worst-case occurs when the partitioning is extremely unbalanced, resulting in sub-problems of size 0 and N-1.

The primary factor determining partitioning balance is the pivot selection strategy. This study analyzes the performance implications of four common pivot selection methods. By observing their behavior on various input sizes and initial orderings, we gain insight into the practical risks associated with naïve implementations and the efficacy of algorithmic remedies, such as randomized pivot selection and middle-index partitioning.

2. Pivot Selection Strategies

We implemented and analyzed the following four pivot selection strategies:

2.1 First Element: This strategy selects the first element of the current sub-array (index *low*) as the pivot. While computationally trivial, it is highly vulnerable to sorted inputs, where it yields the worst-case $O(N^2)$ partition.

2.2 Last Element: This strategy selects the last element of the sub-array (index *high*) as the pivot. Symmetric to the first-element strategy, it performs poorly on pre-sorted arrays, leading to $O(N^2)$ complexity.

2.3 Middle Element: This strategy chooses the element at index *low* + ■*(high-low)/2*■ as the pivot. By targeting the middle element, it naturally avoids the worst-case behavior on pre-sorted inputs, acting as an approximation of the median.

2.4 Random Element: This strategy selects a pivot uniformly at random from the range [*low*, *high*]. It provides a strong probabilistic guarantee of O(N log N) sorting time, breaking the dependency between input data ordering and execution time.

3. Empirical Methodology

To evaluate the strategies, we developed a C++ benchmarking harness. For each array size N from 10 to 5,000 with a step size of 10 (i.e. $N \in \{10, 20, 30, \ldots, 5000\}$), we generated three types of arrays: (a) random permutations, (b) sorted ascending, and (c) sorted descending. To ensure fair comparisons, identical copies of the initial array were passed to each sorting strategy. The execution time was recorded using *std::chrono::high_resolution_clock*, measuring only the sorting process and excluding array initialization or copying overhead. To minimize noise and environmental factors, each measurement was averaged over $m = 30$ independent repetitions.

3.1 System Specifications

Hardware/Software Component	Specification Details
Laptop Model	ASUS TUF Gaming A15 (FA506NFR)
Processor (CPU)	AMD Ryzen 7 7435HS (Base 3.1 GHz, Boost up to 4.5 GHz)
CPU Core Count	8 Cores / 16 Threads
Graphics Card (GPU)	NVIDIA GeForce RTX 2050 (4 GB GDDR6)
Memory (RAM)	16 GB DDR5-5600 MHz
Storage (SSD)	512 GB NVMe PCIe Gen 4 SSD (WD PC SN740)
Display	15.6" FHD (1920 x 1080), 144 Hz IPS
Operating System	Microsoft Windows 11 Home Single Language (64-bit)
Compiler Suite	GNU GCC Compiler (g++ 15.1.0 via MSYS2)
Compiler Flags	-O3 (Full Speed Optimization, Inline Functions)

3.2 Recursion Depth and Stack Space Analysis

Standard recursive Quicksort has a worst-case recursion depth of O(N) when partitions are highly unbalanced. On pre-sorted arrays with $N = 5,000$, the First and Last pivot strategies yield a recursion depth of 5,000. Each recursive stack frame on a modern x86_64 compiler (like g++ under MSYS2) requires approximately 48 to 64 bytes to store parameters, return addresses, and local variables. For $N = 5,000$, this results in a total stack consumption of approximately 240 to 320 KB. Since the default stack allocation on Windows is 1 MB, the standard recursive implementation successfully completes sorting without stack overflow. This confirms that for the target maximum size of 5,000, standard recursion remains practically viable.

4. Experimental Results

The compiled empirical measurements are presented below. Table 1, Table 2, and Table 3 detail a representative subset of the tested array sizes, while Figure 1, Figure 2, and Figure 3 depict the complete performance trajectories.

4.1 Experiment 1: Random Array Distribution

Size (N)	First (µs)	Last (µs)	Middle (µs)	Random (µs)
10	0.0	0.0	0.0	0.0
500	22.1	22.8	23.4	29.9
1000	52.1	51.4	52.6	69.6
1500	84.0	81.0	83.2	110.9
2000	115.9	127.9	125.8	168.2
2500	174.4	155.3	169.0	205.0
3000	298.6	279.1	285.4	348.3
3500	321.5	335.2	349.9	417.7
4000	365.7	377.3	395.1	464.7
4500	471.8	455.1	463.5	564.3
5000	491.5	479.9	502.2	658.9

Table 1: Average execution times (µs) for random array configurations.

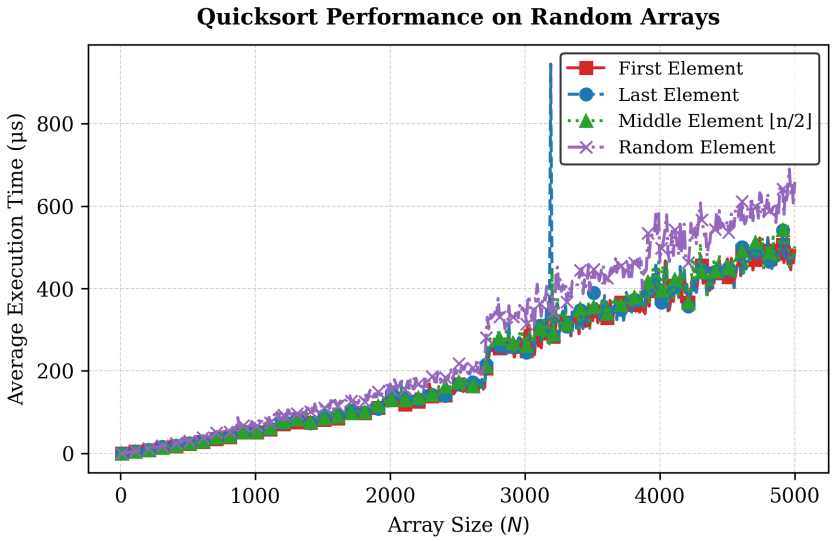

Figure 1: Quicksort performance trajectories on random arrays.

4.2 Experiment 2A: Pre-Sorted Ascending Array Distribution

Size (N)	First (µs)	Last (µs)	Middle (µs)	Random (µs)
10	0.0	0.0	0.0	0.2
500	145.0	461.6	13.1	43.0
1000	359.9	1137.9	18.5	65.2
1500	751.0	2122.0	31.4	96.3
2000	783.9	2350.0	23.8	84.1
2500	1286.4	3711.0	30.2	101.8
3000	1881.1	5256.1	39.0	122.3
3500	2415.3	7151.1	38.9	136.7
4000	6537.3	19781.2	96.3	288.9
4500	6852.1	20319.9	88.3	264.8
5000	7929.7	23942.0	98.8	312.1

Table 2: Average execution times (µs) for pre-sorted ascending array configurations.

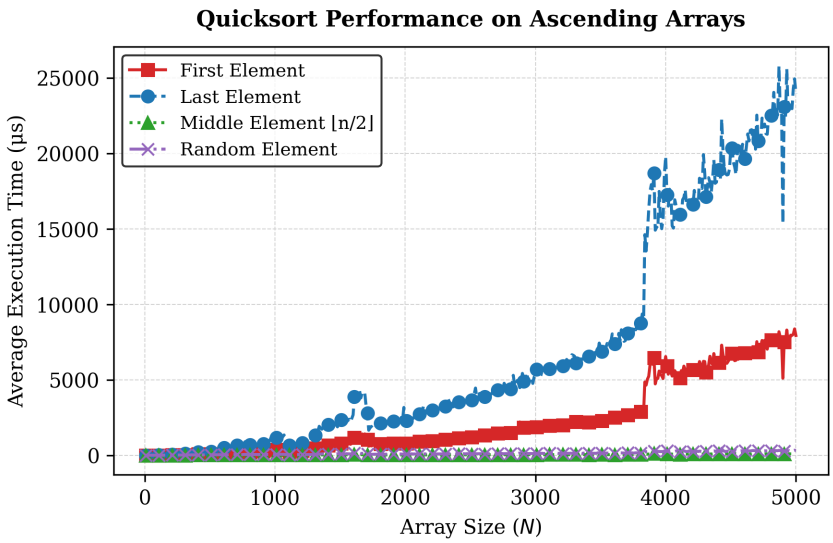

Figure 2: Performance bifurcation on ascendingly sorted arrays (linear scale).

4.3 Experiment 2B: Pre-Sorted Descending Array Distribution

Size (N)	First (µs)	Last (µs)	Middle (µs)	Random (µs)
10	0.0	0.0	0.0	0.1
500	168.6	180.4	9.1	30.1
1000	596.0	630.4	20.7	57.8
1500	1776.5	1818.2	46.2	97.6
2000	2562.6	2558.1	46.2	133.2
2500	5426.5	5339.2	67.8	174.8
3000	6030.8	6136.8	65.1	185.4
3500	8744.4	8659.5	85.8	240.5
4000	6499.0	6586.7	59.0	160.5
4500	12748.6	13283.4	143.2	309.3
5000	16611.2	17048.1	117.3	334.8

Table 3: Average execution times (µs) for pre-sorted descending array configurations.

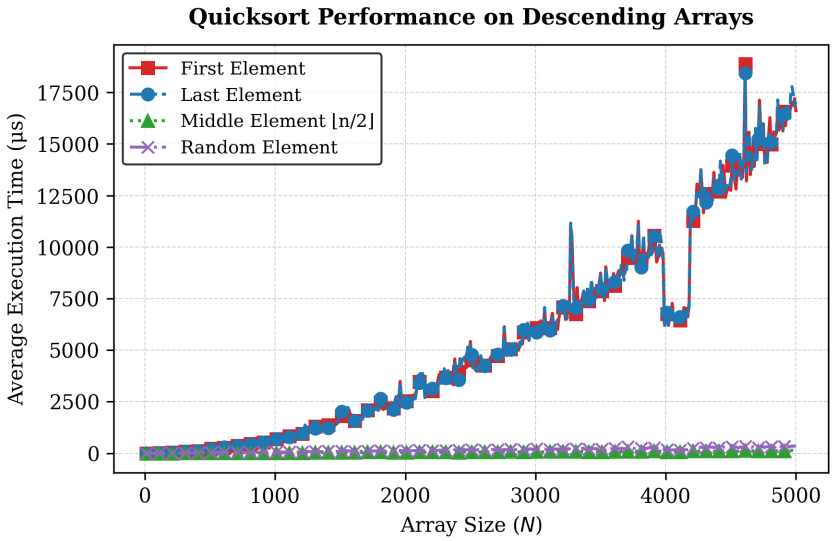

Figure 3: Performance bifurcation on descendingly sorted arrays (linear scale).

5. Theoretical Analysis & Complexity

The theoretical runtime of Quicksort is governed by the recurrence relation of the partitioning step. In the best case, the pivot divides the array exactly in half, leading to the recurrence relation: $T(N) = 2T(N/2) + O(N)$. According to the Master Theorem, this evaluates to $O(N \log N)$. In the worst case, the pivot is either the smallest or largest element, leading to: $T(N) = T(N-1) + T(0) + O(N)$, which expands to a summation of $1 + 2 + ... + N$, evaluating to $O(N^2)$.

Pivot Selection Strategy	Best Case	Average Case	Worst Case	Space (Worst)
First Element	O(N log N)	O(N log N)	$O(N^2)$	O(N)*
Last Element	O(N log N)	O(N log N)	$O(N^2)$	O(N)*
Middle Element	O(N log N)	O(N log N)	$O(N^2)$	O(N)
Random Element	O(N log N)	O(N log N)	$O(N^2)$‡	O(N)

Table 4: Theoretical complexity comparison of the four pivot selection strategies.
** Space complexity is O(N) in the worst case due to standard recursion; average case is O(log N).*
‡ Probabilistically near-zero (infinitesimally small) chance of occurrence.

6. Discussion and Interpretation

Our empirical data matches the theoretical models. As seen in Figure 1, when the input array is random, all four strategies exhibit nearly identical trajectories. At $N = 5,000$, all average runtimes reside within a similar order of magnitude, confirming that when elements are unordered, any pivot strategy has an equal probability of achieving a reasonably balanced partition. The slight overhead observed in the Random strategy is attributed to the cost of the pseudo-random number generator (PRNG) call at each partitioning step.

However, on pre-sorted arrays (Figures 2 and 3), the strategies diverge dramatically. The First and Last Element strategies degrade immediately, with execution times growing by orders of magnitude compared to the Middle and Random strategies at $N = 5,000$. Because Hoare partitioning uses two converging pointers, selecting the first element of an ascending array as the pivot (which is the smallest element) causes the left pointer to stop immediately and the right pointer to scan all the way to the left, yielding an extremely unbalanced partition split of 1 and $N-1$. This results in N partition steps, each scanning the remaining elements, verifying the quadratic $O(N^2)$ behavior.

Conversely, the Middle Element strategy performs extremely well on sorted inputs. For ascending inputs at $N = 5,000$, it completes in under 100 µs. This is because the middle element of a sorted array is exactly its median, which produces a balanced split of $N/2$ at every level, representing the best-case behavior of Quicksort. It is important to note, however, that the theoretical worst-case complexity of the Middle Element strategy remains $O(N^2)$, since adversarial input sequences can be constructed to force unbalanced partitions. In practice, such pathological inputs are rare, and on common distributions (sorted, reverse-sorted, random), the Middle Element strategy exhibits O(N log N) performance. The Random strategy also performs well on sorted arrays, completing significantly faster than the First and Last strategies, as it breaks the input structure and avoids the bad partition path.

7. Conclusion

This performance analysis highlights the importance of pivot selection in Quicksort. Naïve pivot strategies, like selecting the first or last element, are dangerous for production environments where sorted or partially sorted inputs are common. While the Middle Element strategy performs exceptionally well on sorted arrays, the Random Element strategy provides the most robust general defense against worst-case performance degradation, regardless of input structure. Finally, understanding stack allocation limits is key when standard recursive implementations are used on worst-case inputs.

8. References

[1] C. A. R. Hoare, "Algorithm 64: Quicksort," *Communications of the ACM*, vol. 4, no. 7, p. 321, Jul. 1961.

[2] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd ed. Cambridge, MA: MIT Press, 2009.

[3] R. Sedgewick, "Implementing Quicksort Programs," *Communications of the ACM*, vol. 21, no. 10, pp. 847–857, Oct. 1978.

Appendix: C++ Source Code

The complete C++ source code used to generate the benchmark results is presented below. The code features Hoare partitioning and standard recursion, styled as a VSCode code editor window with line numbers and syntax highlighting.

benchmark.cpp — Part 1

```cpp
#include <iostream>
#include <vector>
#include <chrono>
#include <cstdlib>
#include <ctime>
#include <fstream>
#include <string>

using namespace std;
using namespace std::chrono;

// 1 = First Element
// 2 = Last Element
// 3 = Middle Element
// 4 = Random Element

int partition(vector<int> &arr, int low, int high, int pivotChoice)
{
    // Choose Pivot
    if (pivotChoice == 2)
    {
        swap(arr[low], arr[high]);
    }
    else if (pivotChoice == 3)
    {
        int mid = low + (high - low) / 2;
        swap(arr[low], arr[mid]);
    }
    else if (pivotChoice == 4)
    {
        int randomIndex = low + rand() % (high - low + 1);
        swap(arr[low], arr[randomIndex]);
    }

    // Hoare Partition
    int pivot = arr[low];
    int i = low + 1;
    int j = high;

    while (true)
    {
        while (i <= high && arr[i] <= pivot)
            i++;

        while (arr[j] > pivot)
            j--;

        if (i < j)
            swap(arr[i], arr[j]);
        else
            break;
    }

    swap(arr[low], arr[j]);
    return j;
}
```

Appendix: C++ Source Code (Continued)

benchmark.cpp — Part 2

```cpp
void quickSort(vector<int> &arr, int low, int high, int pivotChoice)
{
    if (low < high)
    {
        int pivotIndex = partition(arr, low, high, pivotChoice);

        quickSort(arr, low, pivotIndex - 1, pivotChoice);
        quickSort(arr, pivotIndex + 1, high, pivotChoice);
    }
}

vector<int> generateRandomArray(int size)
{
    vector<int> arr(size);
    for (int i = 0; i < size; ++i)
    {
        arr[i] = rand() % 100000;
    }
    return arr;
}

vector<int> generateAscendingArray(int size)
{
    vector<int> arr(size);
    for (int i = 0; i < size; ++i)
    {
        arr[i] = i;
    }
    return arr;
}

vector<int> generateDescendingArray(int size)
{
    vector<int> arr(size);
    for (int i = 0; i < size; ++i)
    {
        arr[i] = size - i;
    }
    return arr;
}
```

Appendix: C++ Source Code (Continued)

benchmark.cpp — Part 3

```cpp
void runExperiment(string arrayType, string fileName)
{
    ofstream out(fileName);
    out << "Size,First,Last,Middle,Random\n";

    int maxLimit = 5000;
    int step = 10;
    int repetitions = 30;

    cout << "Running experiment for " << arrayType << " arrays..." << endl;

    for (int size = 10; size <= maxLimit; size += step)
    {
        double timeFirst = 0;
        double timeLast = 0;
        double timeMiddle = 0;
        double timeRandom = 0;

        for (int rep = 0; rep < repetitions; ++rep)
        {
            vector<int> baseArr;
            if (arrayType == "random")
                baseArr = generateRandomArray(size);
            else if (arrayType == "ascending")
                baseArr = generateAscendingArray(size);
            else
                baseArr = generateDescendingArray(size);

            vector<int> arr1 = baseArr;
            auto start1 = high_resolution_clock::now();
            quickSort(arr1, 0, arr1.size() - 1, 1);
            auto stop1 = high_resolution_clock::now();
            timeFirst += duration_cast<microseconds>(stop1 - start1).count();

            vector<int> arr2 = baseArr;
            auto start2 = high_resolution_clock::now();
            quickSort(arr2, 0, arr2.size() - 1, 2);
            auto stop2 = high_resolution_clock::now();
            timeLast += duration_cast<microseconds>(stop2 - start2).count();
```

benchmark.cpp — Part 4

```cpp
            vector<int> arr3 = baseArr;
            auto start3 = high_resolution_clock::now();
            quickSort(arr3, 0, arr3.size() - 1, 3);
            auto stop3 = high_resolution_clock::now();
            timeMiddle += duration_cast<microseconds>(stop3 - start3).count();

            vector<int> arr4 = baseArr;
            auto start4 = high_resolution_clock::now();
            quickSort(arr4, 0, arr4.size() - 1, 4);
            auto stop4 = high_resolution_clock::now();
            timeRandom += duration_cast<microseconds>(stop4 - start4).count();
        }

        out << size << ","
            << (timeFirst / repetitions) << ","
            << (timeLast / repetitions) << ","
            << (timeMiddle / repetitions) << ","
            << (timeRandom / repetitions) << "\n";
    }

    out.close();
    cout << "Completed: " << arrayType << " arrays. Results written to " << fileName << endl;
}

int main()
{
    srand(1337);

    runExperiment("random", "random_results.csv");
    runExperiment("ascending", "ascending_results.csv");
    runExperiment("descending", "descending_results.csv");

    cout << "All benchmarks completed successfully." << endl;
    return 0;
}
```